Lecture 10

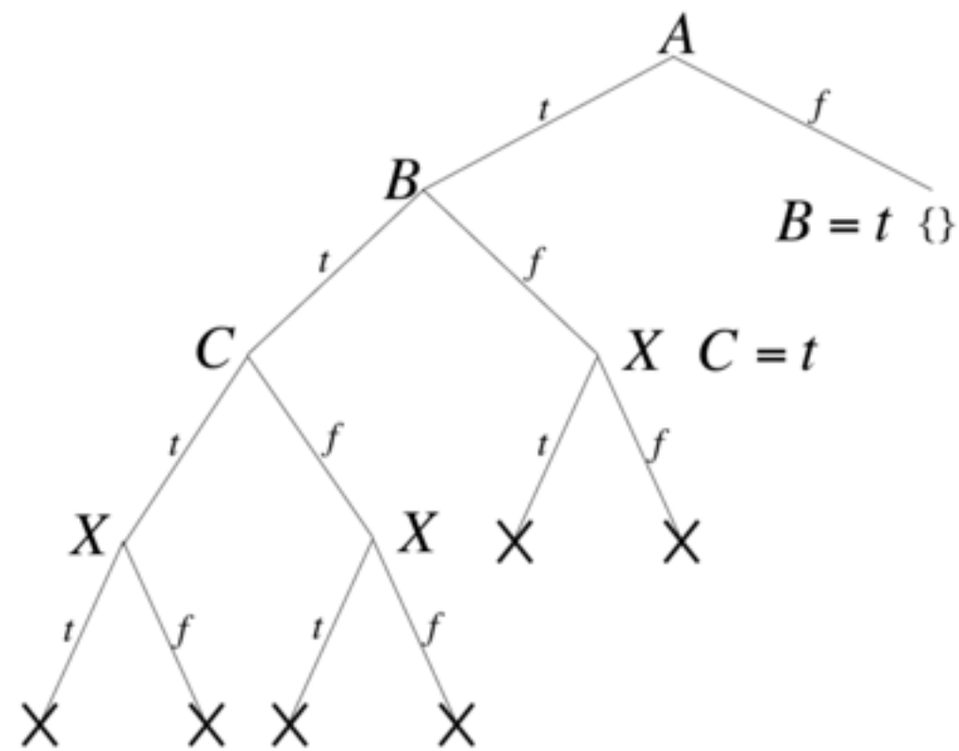
Pete Manolios
Northeastern

DPLL SAT Algorithm

- BCP
- Base case: empty clause: UNSAT
- Remove clauses containing pure literals (modern solvers don't do this)
- Base case: no clauses: SAT
- Choose some var, say x
  (if removing pure literals, x has to appear in both phases)
  - Add {x} and recursively call DPLL
  - Add {¬x} and recursively call DPLL
  - If one of the calls returns SAT, return SAT
  - Else return UNSAT
- Correctness follows from Shannon expansion
- In contrast to DP, space is not a problem

DPLL SAT Example

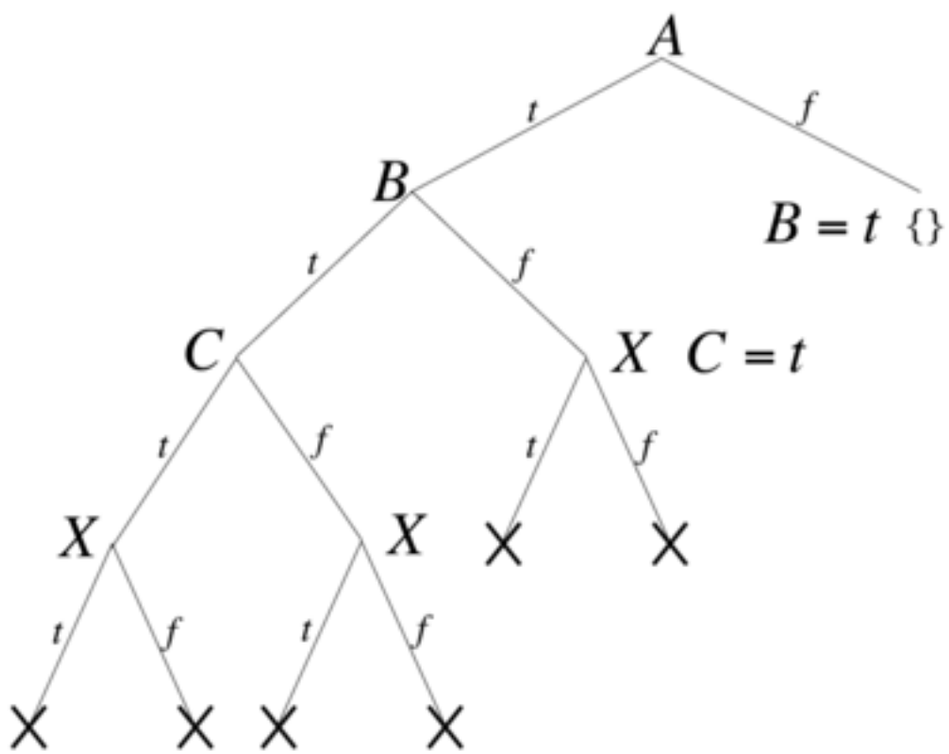
$$\Delta = \begin{array}{l} 1.\ \{A, B\} \\ 2.\ \{B, C\} \\ 3.\ \{\neg A, \neg X, Y\} \\ 4.\ \{\neg A, X, Z\} \\ 5.\ \{\neg A, \neg Y, Z\} \\ 6.\ \{\neg A, X, \neg Z\} \\ 7.\ \{\neg A, \neg Y, \neg Z\} \end{array}$$



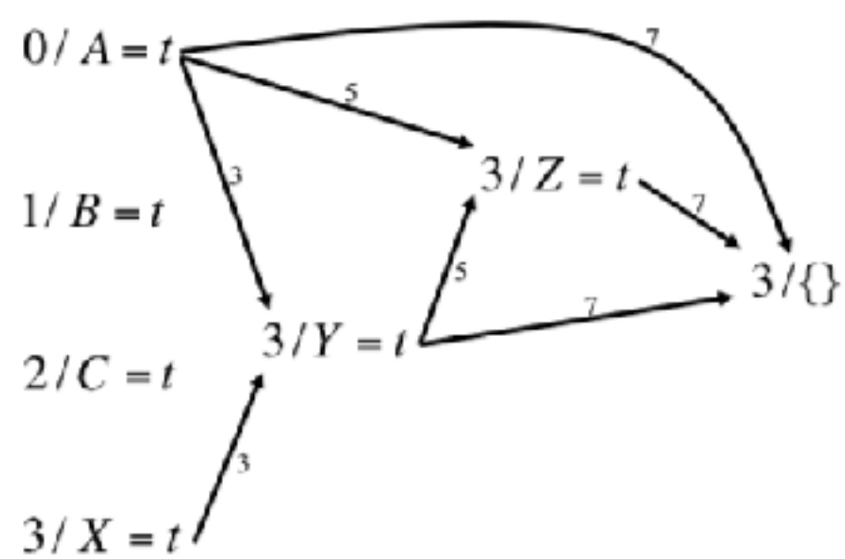
- Note that when DPLL detects contradictions it backtracks chronologically
- When we get a contradiction with X, we try ¬X, then we go back and try ¬C and X, ¬X again, …
- But the real problem was that we set A; can we avoid this exponential search?
- Yes: non-chronological backtracking, a major improvement

Examples/figures from chp. 3 SAT handbook: pure literals not removed

Implication Graphs

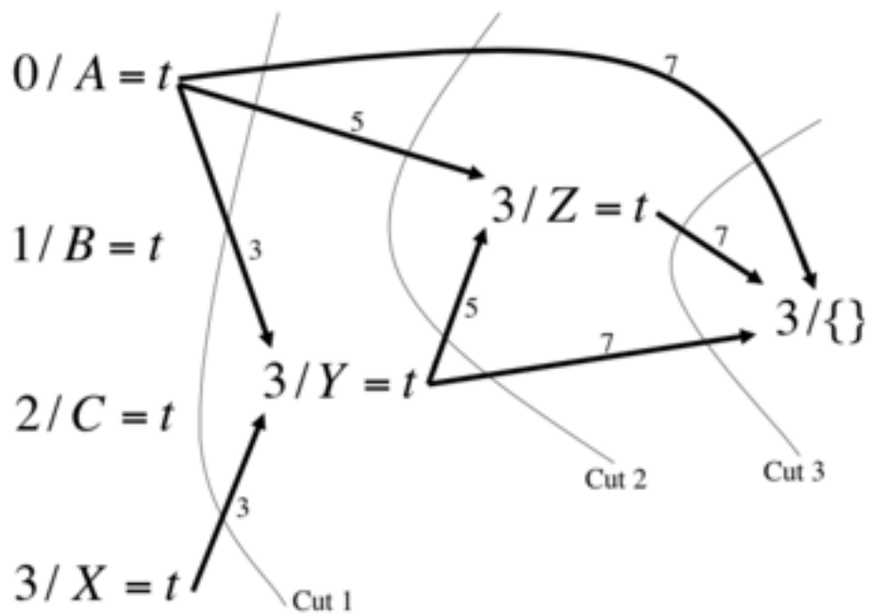


$$\Delta = \begin{array}{l} 1.\ \{A, B\} \\ 2.\ \{B, C\} \\ 3.\ \{\neg A, \neg X, Y\} \\ 4.\ \{\neg A, X, Z\} \\ 5.\ \{\neg A, \neg Y, Z\} \\ 6.\ \{\neg A, X, \neg Z\} \\ 7.\ \{\neg A, \neg Y, \neg Z\} \end{array}$$



- Nodes are l/V=v: var V set to v @ level l
- If node implied, justification recorded (clause #, edges from assignments)
- {} denotes contradiction

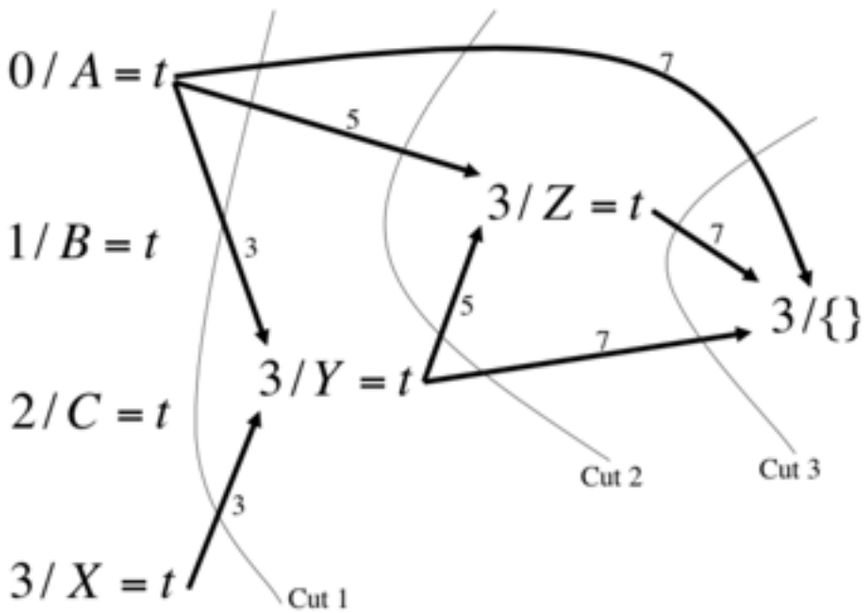
Conflict-Driven Clauses



$$\Delta = \begin{array}{l} 1.\ \{A, B\} \\ 2.\ \{B, C\} \\ 3.\ \{\neg A, \neg X, Y\} \\ 4.\ \{\neg A, X, Z\} \\ 5.\ \{\neg A, \neg Y, Z\} \\ 6.\ \{\neg A, X, \neg Z\} \\ 7.\ \{\neg A, \neg Y, \neg Z\} \end{array}$$

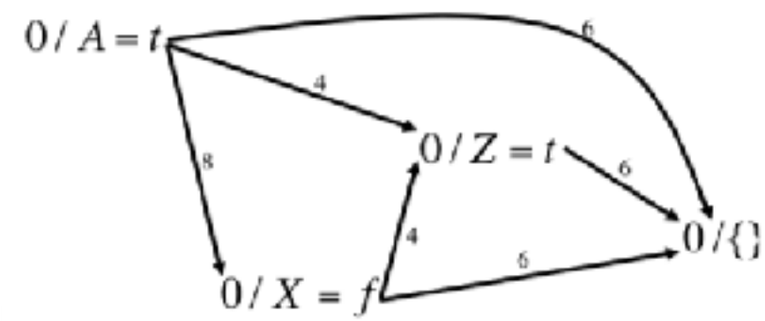
- Consider any cut of the implication graph that separates decision vars from {}
- The nodes with an edge that crosses the cut are in conflict set
- Negate the assignments in the set to obtain a conflict-driven clause
- Conflict clauses: Cut1: {¬A,¬X}, Cut2: {¬A, ¬Y}, Cut3: {¬A, ¬Z, ¬Y}
- Conflict–driven clauses generated from cuts that contain exactly one variable assigned at the level of conflict are said to be asserting: Cut1 & Cut2 (not Cut 3)

Non-Chronological Backtracking



$\Delta =$

1. $\{A, B\}$
2. $\{B, C\}$
3. $\{\neg A, \neg X, Y\}$
4. $\{\neg A, X, Z\}$
5. $\{\neg A, \neg Y, Z\}$
6. $\{\neg A, X, \neg Z\}$
7. $\{\neg A, \neg Y, \neg Z\}$



- Asserting conflict clauses: Cut1: 8. {¬A,¬X}, Cut2: {¬A, ¬Y}
- Assertion level: 2nd highest level in asserting clause (0 for cuts 1, 2) or -1
- Backtrack to assertion level and add a learned clause (non-chronological!)
- We can now immediately infer (BCP) ¬X (we use Cut1), so we have A, ¬X
- Then by BCP: Z (4), ¬Z (6) so we get a new implication graph
- Asserting clauses: {¬A} at level -1, so we have ¬A, BCP: B and we're done
- Compare to previous search, where the algorithm had to go back a level at a time
- Clause learning can generate exponentially shorter proofs of unsat!

Modern CDCL Solvers

- Based on DPLL, but with conflict-driven clause learning
- Data structures to speed up BCP: 2-watched literal scheme
- Data structures for clause learning
- Decision heuristics: select recently active literals (VSIDS)
- Preprocessing: greedy variable elimination
- Inprocessing: interleave preprocessing & search
- Clause deletion: learned clauses lead to memory & efficiency problems, so delete large, inactive clauses
- Random restarts: keep learned clauses, but restart
  - avoids getting stuck in hard part of search space
  - phase saving: pick last phase of assignment

HornSAT

- A CNF formula is Horn if every clause has at most one positive literal
  - (¬a,b), (¬a,¬b,¬c,¬d), (a),(¬b,¬a,d),(¬c)
- Think of clauses as rules that “fire” under assignment A, if LHS holds
  - a⇒b, abcd⇒false, a, ba⇒d, c⇒false (or ¬c)
- HornSAT is in P
  - BCP (until fixpoint), constructing a partial assignment
  - If empty clause, return unsat
  - Else return sat (set remaining vars to false)
- Minimal assignment returned: all lits that have to be true in all sat assignments
- Note: if all clauses have 1 pos literal, then SAT (assign true to every var)
- Linear time: BCP
- Dual horn: every clause has at most one negative literal (same problem)
- Connection with forward chaining in ACL2s/Datalog
  - Start with known facts; apply forward-chaining rules; repeat until fixpoint reached
  - Datalog: sets of Horn clauses over relational predicates. Apply forward chaining until fixpoint

Renamable Horn

- What about {x, y, ¬z}, {¬x, y, ¬z}?
- Renamable horn: There is a subset of variables such that if we negate every occurrence, we have a horn formula
- Can determine if renamable horn in Ptime
- Can solve such problems in Ptime
- Lemma: F is renamable Horn iff there exists an interpretation such that at most one literal per clause is false
- So, we can test for renamability of F by using 2SAT
  - Find sat. assignment for $\wedge c \in F \wedge u,v \in c \ (u \vee v)$
  - Rename variables occurring positively in assignment
- Unit propagation can solve renamable horn problems (and more) in linear time (so no need to check for renamability)

First Order Logic

- Example: Group Theory
  - (G1) For all x, y, z: $(x \cdot y) \cdot z = x \cdot (y \cdot z)$
  - (G2) For all x: $x \cdot e = x$
  - (G3) For all x there is a y such that: $x \cdot y = e$
- Theorem: For every x, there is a y such that $y \cdot x = e$
- Examples of groups: Nat, +, 0?; Int, +, 0?, Real, *, 1?
- Proof:

  By (G3) there is: a y s.t. $x \cdot y = e$ and a z s.t. $y \cdot z = e$

  Now: $y \cdot x = y \cdot x \cdot e = y \cdot x \cdot y \cdot z = y \cdot e \cdot z = y \cdot z = e$
- Is this true for all groups? Why?
- How many groups are there?
- Are there true statements about groups with no proof?

First Order Logic

- First Order Logic forms the foundation of mathematics
- We study various objects, e.g., groups
- Properties of objects captured by “non-logical” axioms
  - (G1-G3 in our example)
- Theory consists of all consequences of “non-logical” axioms
  - Derivable via logical reasoning alone
  - That’s it; no appeals to intuition
- Separation into non-logical axioms logical reasoning is astonishing: all theories use exactly same reasoning
- But, what is a proof ($\Phi \vdash \phi$)?
- Question leads to computer science
- Proof should be so clear, even a machine can check it

First Order Logic: Syntax

- Every FOL (first order language) includes
  - Variables v_0, v_1, v_2, ...
  - Boolean connectives: $\vee$, $\neg$
  - Equality: =
  - Parenthesis: (,)
  - Quantifiers: $\exists$
- The symbol set of a FOL contains (possibly empty) sets of
  - relation symbols, each with an arity > 0
  - function symbols, each with an arity > 0
  - constant symbols
- Example: groups 2-ary function symbol • and constant e
- Set theory: $\in$, a 2-ary relation symbol, ...

First Order Logic: Terms

- Terms denote objects of study, e.g., group elements
- The set of S-terms is the least set closed under:
  - Every variable is a term
  - Every constant is a term
  - If $t_1, \ldots, t_n$ are terms and f is an n-ary function symbol, then $f(t_1, \ldots, t_n)$ is a term

First Order Logic: Formulas

- Formulas: statements about the objects of study
- An atomic formula of S is
  - $t_1 = t_2$ or
  - $R(t_1, ..., t_n)$, where t_i is an S-term and R is an n-ary relation symbol in S
- The set of S-formulas is the least set closed under:
  - Every atomic formula is a formula
  - If ϕ, ψ are S-formulas and x is a variable, then $\neg\phi$, $(\phi \vee \psi)$, and $\exists x\phi$ are S-formulas
- All Boolean connectives can be defined in terms of $\neg$ and $\vee$
- We can define $\forall x\phi$ to be $\neg\exists x\neg\phi$

Definitions on Terms & Formulas

- Define the notion of a free variable for an S-formula
- The definition of formula depends on that of term
- So, we're going to need an auxiliary definition:

  $var(x) = \{x\}$

  $var(c) = \{\}$

  $var(f(t_1, ..., t_n)) = var(t_1) \cup \cdots \cup var(t_n)$
- Is this a definition? (termination!)

  $free(t_1 = t_2) = var(t_1) \cup var(t_2)$

  $free(R(t_1, ..., t_n)) = var(t_1) \cup \cdots \cup var(t_n)$

  $free(\neg\phi) = free(\phi)$

  $free((\phi \vee \psi)) = free(\phi) \cup free(\psi)$

  $free(\exists x\phi) = free(\phi) \setminus \{x\}$